📘 JOBSHEET PRAKTIKUM

Mata Kuliah: Pemrograman Framework

Pertemuan: Implementasi Login Database & Multi-Role

Topik: Sistem Autentikasi Login Terintegrasi Database

🎯 A. Capaian Pembelajaran (CPMK)

Mahasiswa mampu:

1. Menghubungkan login dengan database.
2. Melakukan verifikasi password menggunakan bcrypt.
3. Membuat custom login page.
4. Mengimplementasikan callback URL redirect.
5. Menerapkan middleware authentication.
6. Menerapkan role-based access control (RBAC).

🧠 B. Dasar Teori

1️⃣ Alur Login Database

User input email + password

↓

NextAuth authorize()

↓

Query user berdasarkan email

↓

bcrypt.compare(password)

↓

Jika cocok → return user

↓

Token & Session dibuat

↓

Redirect sesuai callbackURL

2️⃣ Role-Based Access Control (RBAC)

Contoh role:

- user
- admin

Tujuan:

- Halaman tertentu hanya bisa diakses admin.
- User biasa tidak bisa masuk halaman admin.

🛠️ C. Langkah Praktikum

🔹 BAGIAN 1 – Custom Login Page

1️⃣ Tambahkan custom page di **NextAuth line 55-57**

15.praktikum-Login_multi_role > my-app > src > pages > api > auth > [...nextauth].ts > authOptions > pages > signIn

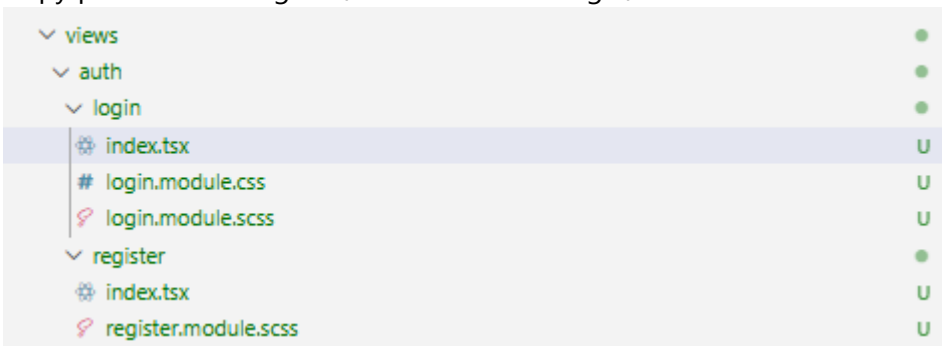
```
1   import NextAuth, { NextAuthOptions } from "next-auth";
2   import CredentialsProvider from "next-auth/providers/credentials";
3
4   export const authOptions: NextAuthOptions = {
5     session: {···
7     },
8     secret: process.env.NEXTAUTH_SECRET,
9     providers: [···
32    ],
33
34    callbacks: {···
53    },
54
55    pages: {
56      signIn: "/auth/login",
57    },
58  };
59  export default NextAuth(authOptions);
60
```

- Jalankan browser http://localhost:3000/ dan klik sign in maka akan diarahkan ke login

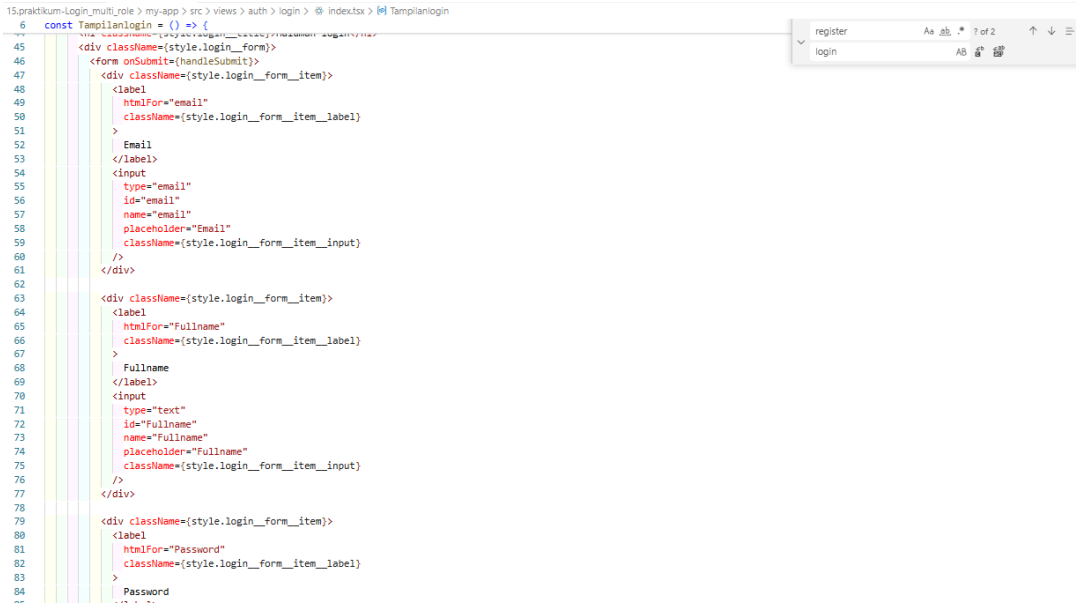
---

BAGIAN 2 – Handle Login di Frontend

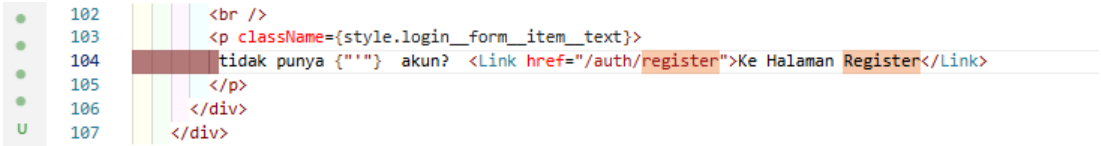
- Copy paste isi dari register/index.tsx ke file login/index.tsx



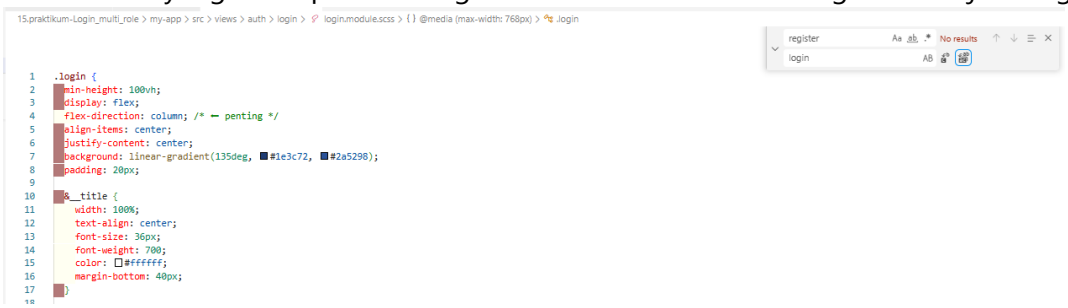
- Copy paste isi dari register/register.module.scss ke file login/login.module.scss
- Semua **text register pada file index.tsx pada folder login diubah menjadi login**



- Jangan lupa setting link hrefnya

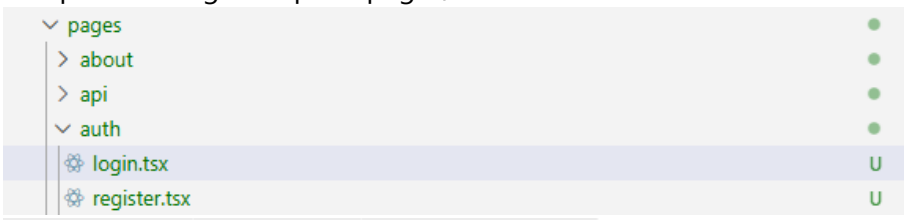


- Lakukan hal yang sama pada file login.module.scss rubah text register menjadi login



```scss
.login {
  min-height: 100vh;
  display: flex;
  flex-direction: column; /* ← penting */
  align-items: center;
  justify-content: center;
  background: linear-gradient(135deg, #1e3c72, #2a5298);
  padding: 20px;

  &__title {
    width: 100%;
    text-align: center;
    font-size: 36px;
    font-weight: 700;
    color: #ffffff;
    margin-bottom: 40px;
  }
```

- Cek pada file login.tsx pada pages/auth



15.praktikum-Login_multi_role > my-app > src > pages > auth > login.tsx > ...

```tsx
import TampilanLogin from "../../views/auth/login";

Windsurf: Refactor | Explain | Generate JSDoc | ×
const halamanLogin = () => {

  return (
    <>
    <TampilanLogin />
    </>
  );
};

export default halamanLogin;
```

- Jalankan browser localhost:3000/auth/login. Tampilannya akan sama dengan register

- Pada tampilan login kita tidak perlu hapus fullname jadi pada folder views/auth/login/index.tsx hapus fullname

15.praktikum-Login_multi_role > my-app > src > views > auth > login > index.tsx > Tampilanlogin

```
6   const Tampilanlogin = () => {
58                placeholder="Email"
59                className={style.login__form__item__input}
60              />
61            </div>
62
63            <div className={style.login__form__item}>
64              <label
65                htmlFor="Fullname"
66                className={style.login__form__item__label}
67              >
68                Fullname
69              </label>
70              <input
71                type="text"
72                id="Fullname"
73                name="Fullname"
74                placeholder="Fullname"
75                className={style.login__form__item__input}
76              />
77            </div>
78
79            <div className={style.login__form__item}>
80              <label
81                htmlFor="Password"
82                className={style.login__form__item__label}
83              >
84                Password
85              </label>
```

Sehingga hasilnya seperti berikut :

- Buka file index.tsx pada folder views/auth/login dan modifikasi codenya seperti berikut (Untuk line 64 sampai kebawah tidak ada perubahan)

15.praktikum-Login_multi_role > my-app > src > views > auth > login > index.tsx > Tampilanlogin > handleSubmit

```tsx
import Link from "next/link";
import style from "../../auth/login/login.module.scss";
import { useState } from "react";
import { useRouter } from "next/router";
import { signIn } from "next-auth/react";

Windsurf: Refactor | Explain | Generate JSDoc | X
const Tampilanlogin = () => {
  const [isLoading, setIsLoading] = useState(false);
  const { push, query } = useRouter();

  const callbackUrl: any = query.callbackUrl || "/";
  const [error, setError] = useState("");
  Windsurf: Refactor | Explain | Generate JSDoc | X
  const handleSubmit = async (event: any) => {
    event.preventDefault();
    setError("");
    setIsLoading(true);

    //const form = event.currentTarget; ···
    // }
    try {
      const res = await signIn("credentials", {
        redirect: false,
        email: event.target.email.value,
        password: event.target.password.value,
        callbackUrl,
      });

      // console.log("signIn response:", res);
      if (!res?.error) {
        setIsLoading(false);
        push(callbackUrl);
      } else {
        setIsLoading(false);
        setError(res?.error || "Login failed");
      }
    } catch (error) {
      setIsLoading(false);
      setError("wrong email or password");
    }
  };
```

Note pastikan tulisan password pada event.password.value pada line 48 sama dengan yang ada di

```tsx
<input
  type="password"
  id="password"
  name="password"
  placeholder="password"
  className={style.login__form__item__input}
/>
```

- Buka file servicefirebase.ts dan tambahkan code di line 25-38

15.praktikum-Login_multi_role > my-app > src > utils > db > TS servicefirebase.ts > signIn

```
1   import {
2     getFirestore,collection,
3     getDocs,Firestore,getDoc,doc,
4     query,addDoc,where,
5   } from "firebase/firestore";
6   import app from "./firebase";
7   import bcrypt from "bcrypt";
8
9   const db = getFirestore(app);
10
    Windsurf: Refactor | Explain | Generate JSDoc | X
11  export async function retrieveProducts(collectionName: string) {···
18  }
19
    Windsurf: Refactor | Explain | Generate JSDoc | X
20  export async function retrieveDataByID(collectionName: string, id: string) {···
24  }
    Windsurf: Refactor | Explain | Generate JSDoc | X
25  export async function signIn(
26    email: string) {
27    const q = query(collection(db, "users"), where("email", "==", email));
28    const querySnapshot = await getDocs(q);
29    const data = querySnapshot.docs.map((doc) => ({
30      id: doc.id,
31      ...doc.data(),
32    }));
33    if (data) {
34      return data[0];
35    }else {
36      return null;
37    }
38  }
    Windsurf: Refactor | Explain | Generate JSDoc | X
39  export async function signUp(···
84  }
85
```

- **BAGIAN 3 – Authorize di NextAuth (Database Login)**
  - Buka file **[...nextauth].ts** modifikasi menjadi berikut **(pada bagian providers)**

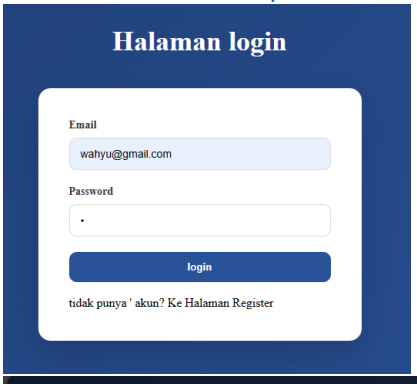
15.praktikum-Login_multi_role > my-app > src > pages > api > auth > TS [...nextauth].ts > ...

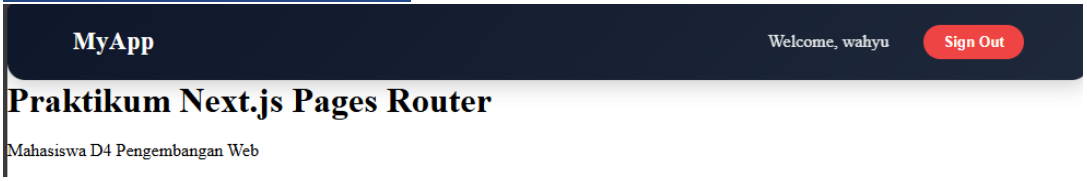
```ts
import { signIn } from "@/utils/db/servicefirebase";
import NextAuth, { NextAuthOptions } from "next-auth";
import CredentialsProvider from "next-auth/providers/credentials";
import bcrypt from "bcrypt";

export const authOptions: NextAuthOptions = {
  session: {
    strategy: "jwt",
  },
  secret: process.env.NEXTAUTH_SECRET,
  providers: [
    CredentialsProvider({
      name: "credentials",
      credentials: {
        // fullname: { label: "Full Name", type: "text" },
        email: { label: "Email", type: "email" },
        password: { label: "Password", type: "password" },
      },
      Windsurf: Refactor | Explain | Generate JSDoc | X
      async authorize(credentials) {
        if (!credentials?.email || !credentials?.password) return null;

        const user: any = await signIn(credentials.email);

        if (user) {
          const isPasswordValid = await bcrypt.compare(
            credentials.password,
            user.password,
          );
          if (isPasswordValid) {
            // Pastikan mengembalikan object user yang bersih
            return {
              id: user.id,
              email: user.email,
              fullname: user.fullname,
              role: user.role,
            };
          }
        }
        return null;
      },
    }),
  ],

```

- BAGIAN 4 – Tambahkan Role ke Token
  - JWT Callback pada file **[...nextauth].ts** Modifikasi menjadi

```
41    callbacks: {
        Windsurf: Refactor | Explain | Generate JSDoc | X
42      async jwt({ token, account, profile, user }: any) {
43        if (account?.provider === "credentials" && user) {
44          token.email = user.email;
45          token.fullname = user.fullname;
46          token.role = user.role;
47        }
48        //   console.log("jwt callback", { token, account, profile, user })
49        return token;
50      },
        Windsurf: Refactor | Explain | Generate JSDoc | X
51      async session({ session, token }: any) {
52        if (token.email) {
53          session.user.email = token.email;
54        }
55        if (token.fullname) {
56          session.user.fullname = token.fullname;
57        }
58        if (token.role) {
59          session.user.role = token.role;
60        }
61        //   console.log("session callback", { session, token })
62        return session;
63      },
64    },
```

  - Jalankan browser http://localhost:3000/auth/login





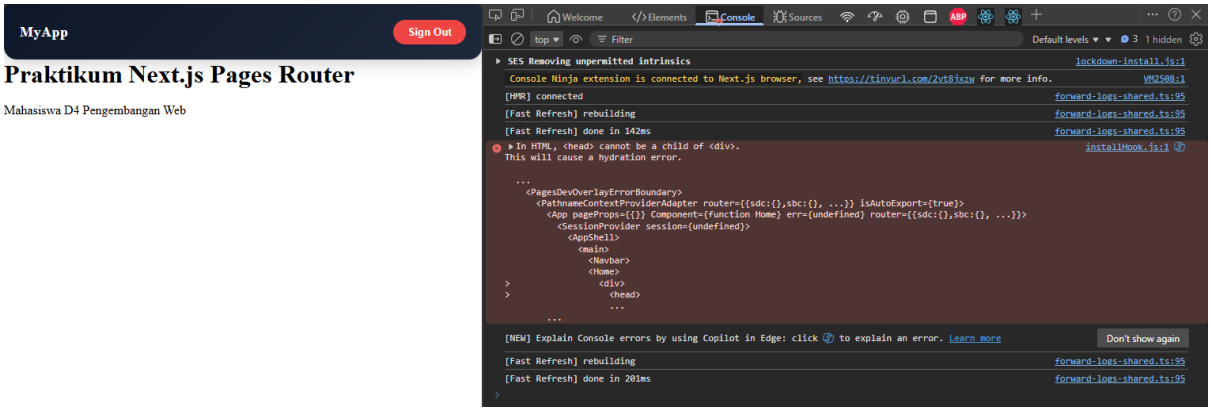
Praktikum Next.js Pages Router

Mahasiswa D4 Pengembangan Web

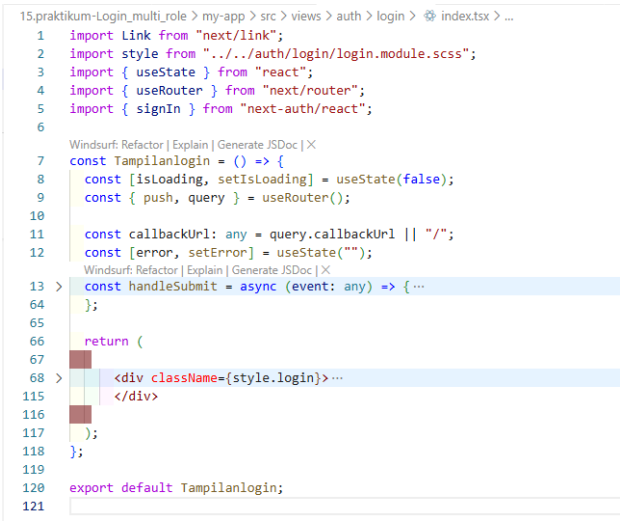
Note ERROR :

JIKA TERDAPAT ERROR SEPERTI INI DISEBABKAN KARENA

In this case, the problem is that a <head> tag is being rendered **inside a** , which is invalid HTML. The <head> element must be a direct child of <html>, not nested inside other elements.



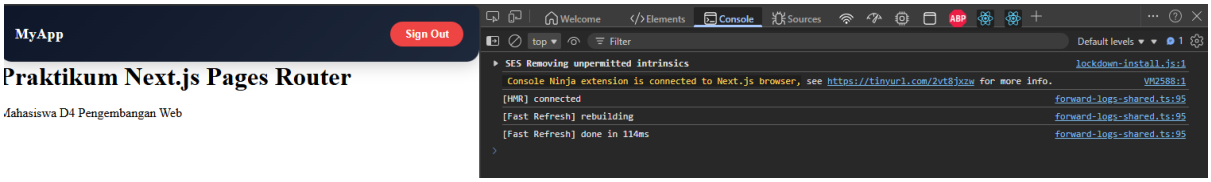
SOLUSINYA BUKA FILE SRC/VIEWS/AUTH/LOGIN/INDEX.TSX



TAMBAHKAN <> </> SEPERTI PADA GAMBAR BERIKUT (LINE 67 DAN 116)

15.praktikum-Login_multi_role > my-app > src > views > auth > login > index.tsx > ...

```tsx
import Link from "next/link";
import style from "../../auth/login/login.module.scss";
import { useState } from "react";
import { useRouter } from "next/router";
import { signIn } from "next-auth/react";

Windsurf: Refactor | Explain | Generate JSDoc | X
const Tampilanlogin = () => {
  const [isLoading, setIsLoading] = useState(false);
  const { push, query } = useRouter();

  const callbackUrl: any = query.callbackUrl || "/";
  const [error, setError] = useState("");
  Windsurf: Refactor | Explain | Generate JSDoc | X
  const handleSubmit = async (event: any) => {···
  };

  return (
    <>
      <div className={style.login}>···
      </div>
    </>
  );
};

export default Tampilanlogin;
```



BAGIAN 5 – Callback URL Logic

- Modifikasi withAuth.ts pada folder src/middleware

15.praktikum-Login_multi_role > my-app > src > Middleware > withAuth.ts > withAuth > <function>

```tsx
import { getToken } from "next-auth/jwt";
import { NextFetchEvent, NextMiddleware, NextRequest, NextResponse } from "next/server";

Windsurf: Refactor | Explain | Generate JSDoc | X
export default function withAuth(
  middleware: NextMiddleware,
  requireAuth: string[] = [],
) {
  return async (req: NextRequest, next: NextFetchEvent) => {
    const pathname = req.nextUrl.pathname;

    if (requireAuth.includes(pathname)) {
      const token = await getToken({
        req,
        secret: process.env.NEXTAUTH_SECRET,
      });
      if (!token) {
        const Url = new URL("/auth/login", req.url);
        Url.searchParams.set("callbackUrl", encodeURI(req.url));
        return NextResponse.redirect(Url);
      }
    }
    return middleware(req, next);
  }
}
```

Tujuannya:
➡️ Setelah login, user kembali ke halaman sebelumnya.

- BAGIAN 6 – Membuat halaman Admin dan authoriz
  - Buat halaman admin

  - Pada index.tsx tambahkan code berikut

15.praktikum-Login_multi_role > my-app > src > pages > admin > index.tsx > ...

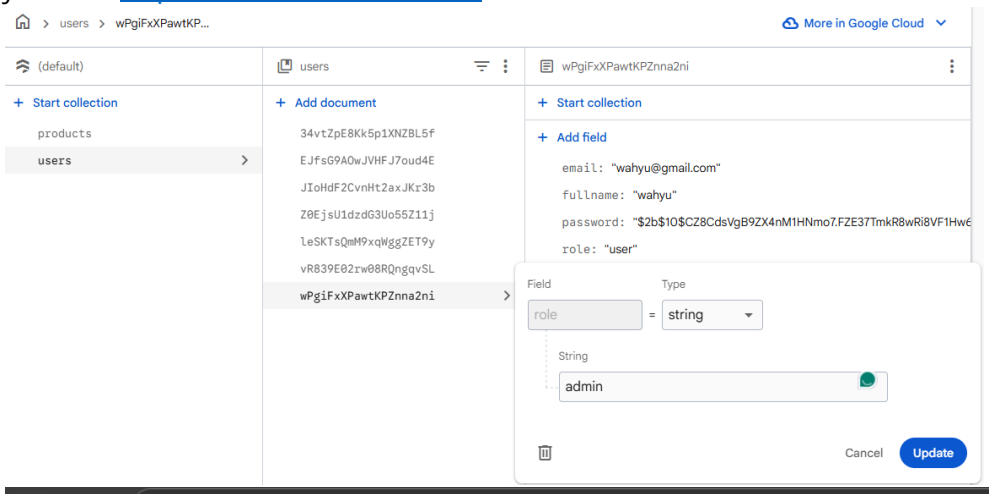
```tsx
Windsurf: Refactor | Explain | Generate JSDoc | ×
const HalamanAdmin = () => {
  return (
    <div>
      <div className="admin">
        <h1>Halaman Admin</h1>
        <p>
          Selamat datang di halaman admin! Anda memiliki akses penuh ke semua
          fitur dan data di aplikasi ini. Di sini, Anda dapat mengelola
          pengguna, melihat laporan, dan melakukan tugas administratif lainnya.
          Pastikan untuk menggunakan hak akses Anda dengan bijak dan menjaga
          keamanan data pengguna.
        </p>
      </div>
    </div>
  );
};

export default HalamanAdmin;

```

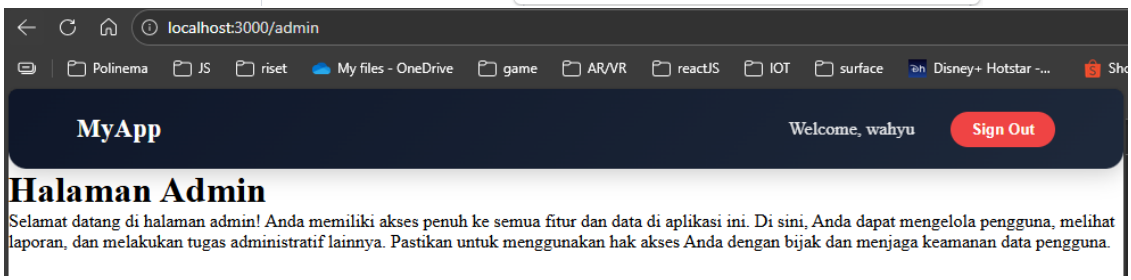
  - Modifikasi withAuth.ts

15.praktikum-Login_multi_role > my-app > src > Middleware > TS withAuth.ts > withAuth

```ts
import { getToken } from "next-auth/jwt";
import { NextFetchEvent, ~~NextMiddleware~~, NextRequest, NextResponse } from "next/server";

const hanyaAdmin = ["/admin"];

Windsurf: Refactor | Explain | Generate JSDoc | ×
export default function withAuth(
  middleware: ~~NextMiddleware~~,
  requireAuth: string[] = [],
) {
  return async (req: NextRequest, next: NextFetchEvent) => {
    const pathname = req.nextUrl.pathname;

    if (requireAuth.includes(pathname)) {
      const token = await getToken({
        req,
        secret: process.env.NEXTAUTH_SECRET,
      });
        if (!token) {
            const Url = new URL("/auth/login", req.url);
            Url.searchParams.set("callbackUrl", encodeURI(req.url));
            return NextResponse.redirect(Url);
        }
        if (token.role !=="admin" && hanyaAdmin.includes(pathname)) {
          return NextResponse.redirect(new URL("/", req.url));
    }
    }
    return middleware(req, next);
  };
}

```

- Jalankan browser localhost:3000/produk dan pada status sudah login. Rubah urlnya menjadi http://localhost:3000/admin maka user akan diarahkan ke localhost. Pada intinya role selain admin tidak bisa mengakses
- Untuk mencoba halaman admin rubah role pada firebas pada salah satu akun dan jalankan http://localhost:3000/admin





🧪 D. Pengujian

🧪 Uji 1 – Login Valid

Input:

- Email benar
- Password benar

Hasil:

- Login berhasil
- Redirect sesuai callbackUrl

🧪 Uji 2 – Password Salah

Input:

- Email benar

- Password salah

Hasil:

- Error message tampil
- Tidak login

🧪 Uji 3 – Akses Admin sebagai User

Login sebagai:

- role: user

Akses:

/admin

Hasil:

- Redirect ke home

🧪 Uji 4 – Akses Admin sebagai Admin

Login sebagai:

- role: admin

Akses:

/admin

Hasil:

- Bisa masuk halaman admin

📊 E. Struktur Database Users

Collection: users

Field	Tipe
email	string
password	string (hashed)
role	string
fullName	string

📊 F. Perbandingan Sistem

Fitur	Sebelum	Sekarang
Login	Hardcoded	Database
Password	Plaintext	Hashed
Role	Tidak ada	Ada
Redirect	Manual	Callback URL
Middleware	Basic	Role-based

🧪 G. Tugas Praktikum

1. Implementasikan login database.
2. Tambahkan role pada user.
3. Buat halaman:
    - /profile
    - /admin
4. Proteksi /admin hanya untuk admin.
5. Implementasikan callback URL.

📝 H. Pertanyaan Analisis

1. Mengapa password harus diverifikasi dengan bcrypt.compare?
2. Mengapa role disimpan di token?
3. Apa fungsi callbackUrl?
4. Mengapa middleware penting untuk security?
5. Apa risiko jika role tidak dicek di middleware?

🏁 I. Output yang Diharapkan

Mahasiswa menghasilkan:

✅ Login terhubung database
✅ Password diverifikasi
✅ Custom login page
✅ Redirect sesuai callback URL
✅ Middleware aktif
✅ Role-based access berjalan

💡 Insight Penting

Level ini sudah masuk kategori:

Production-ready authentication system

Yang sudah mencakup:

- Authentication
- Authorization
- Session management
- Role-based access control